NDTwin Network Digital Twin

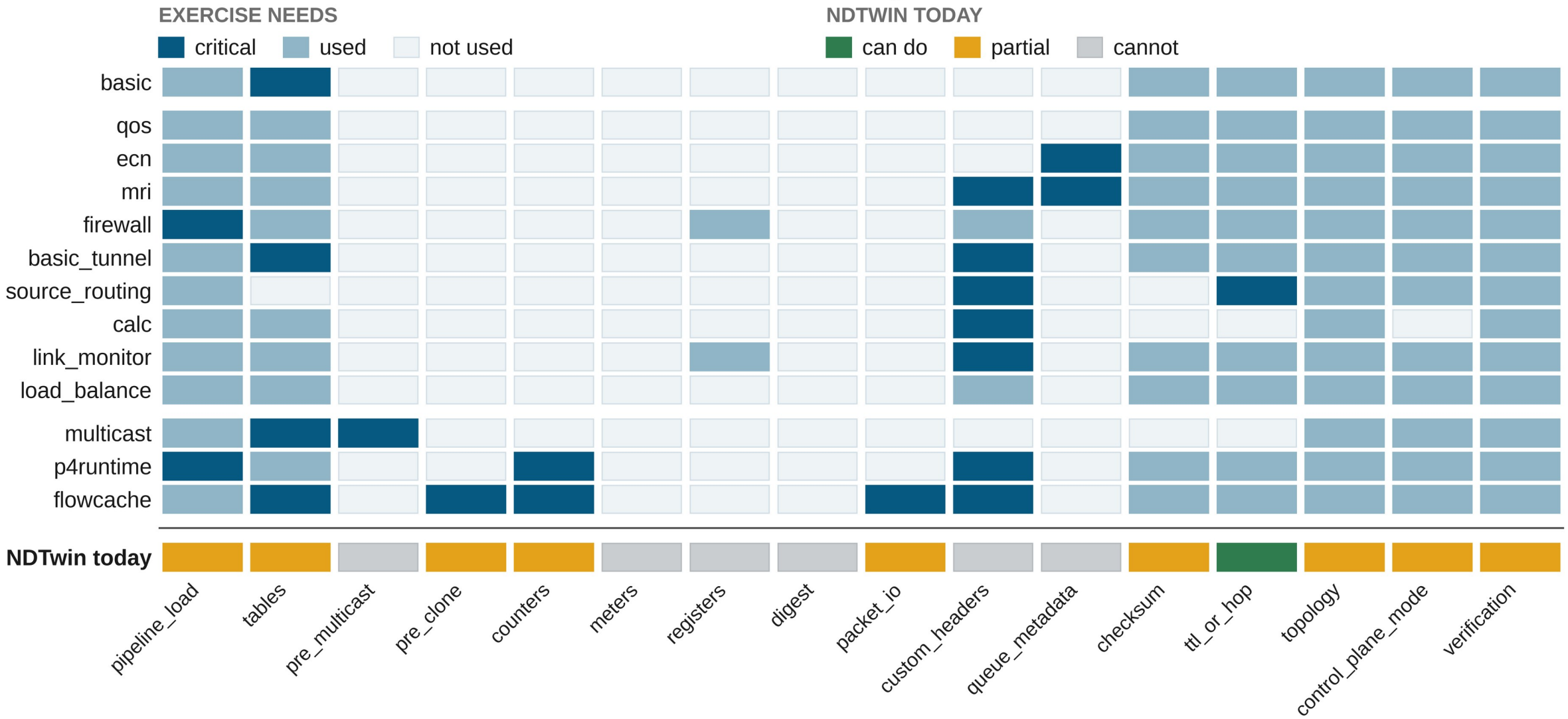
Let the Outside Decide What to Test

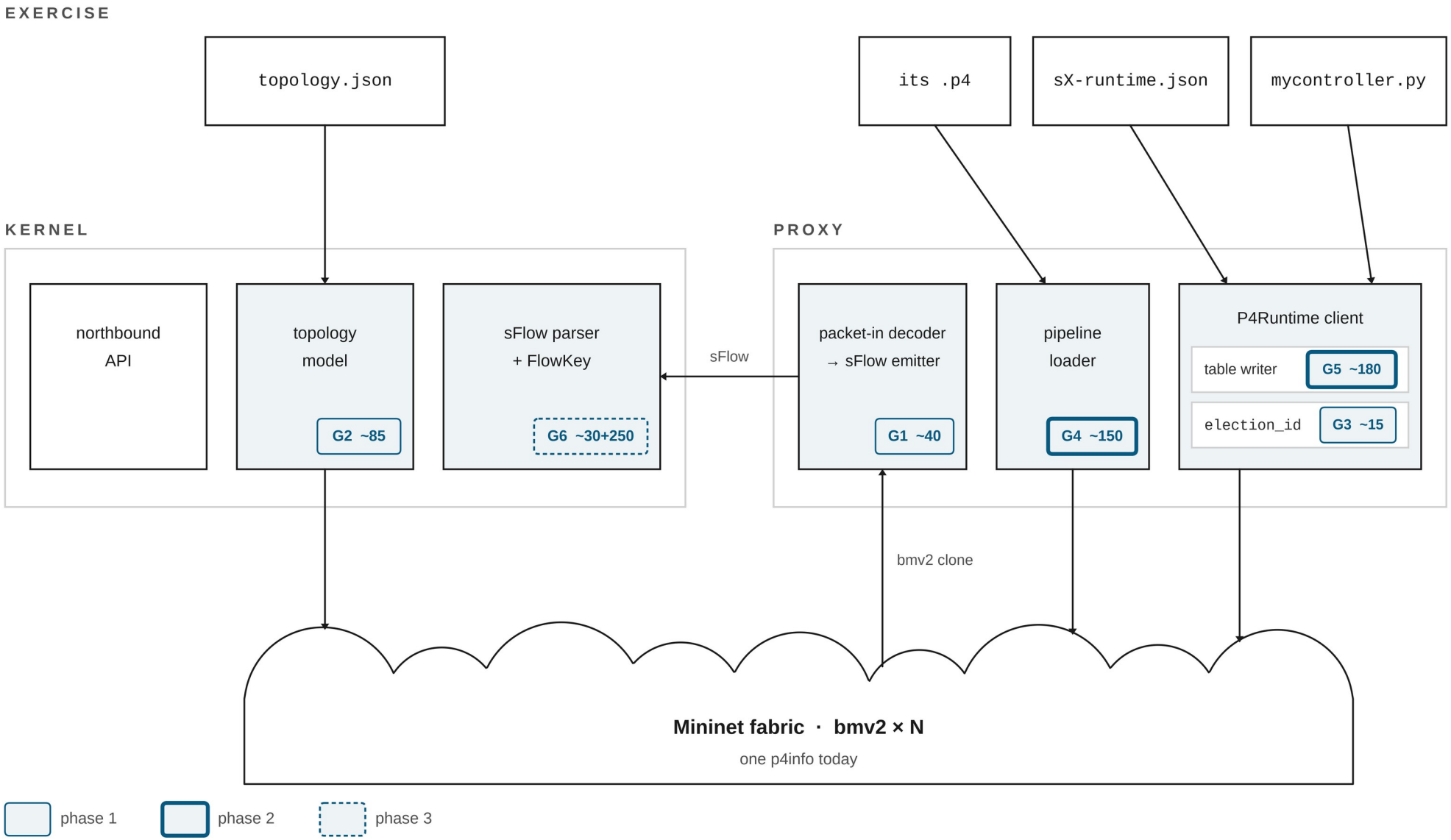
Adam · 2026-09-16

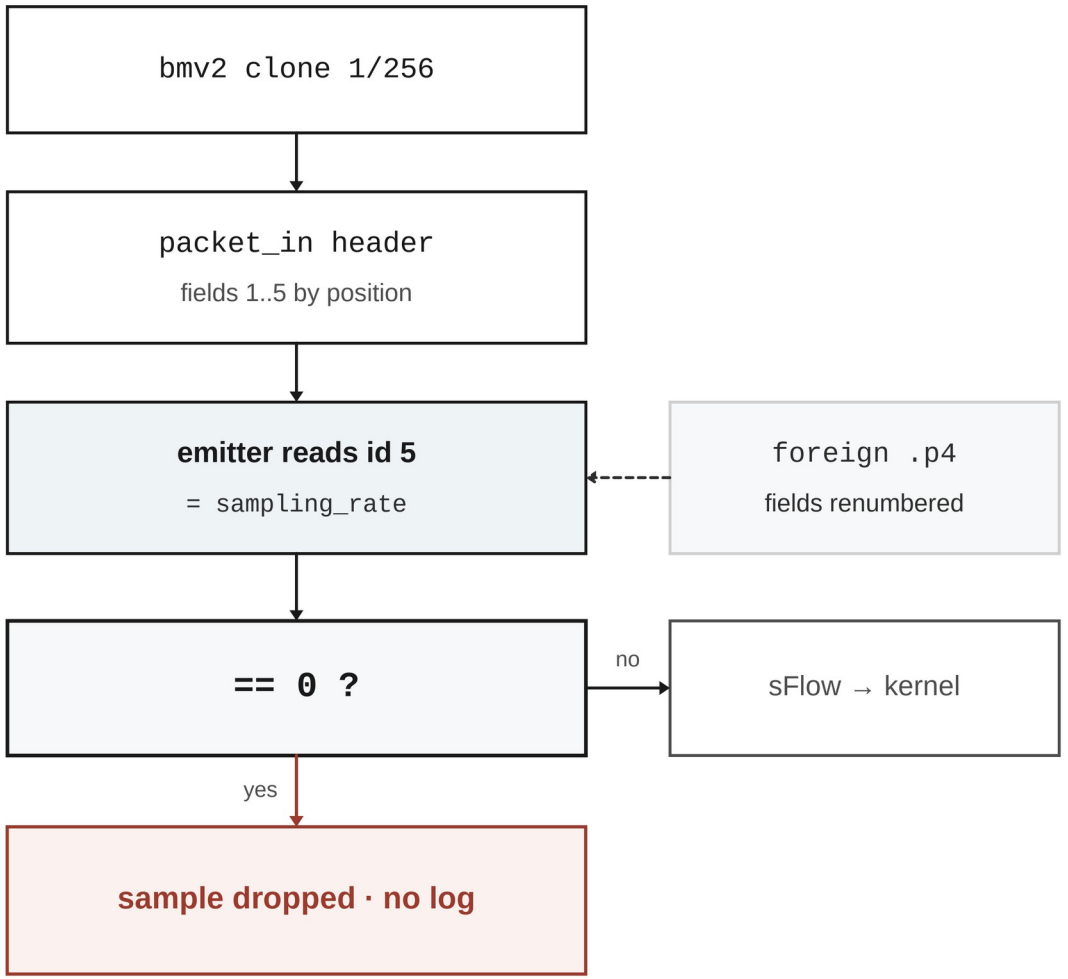
REQUIREMENTS DRIVE THE TESTS

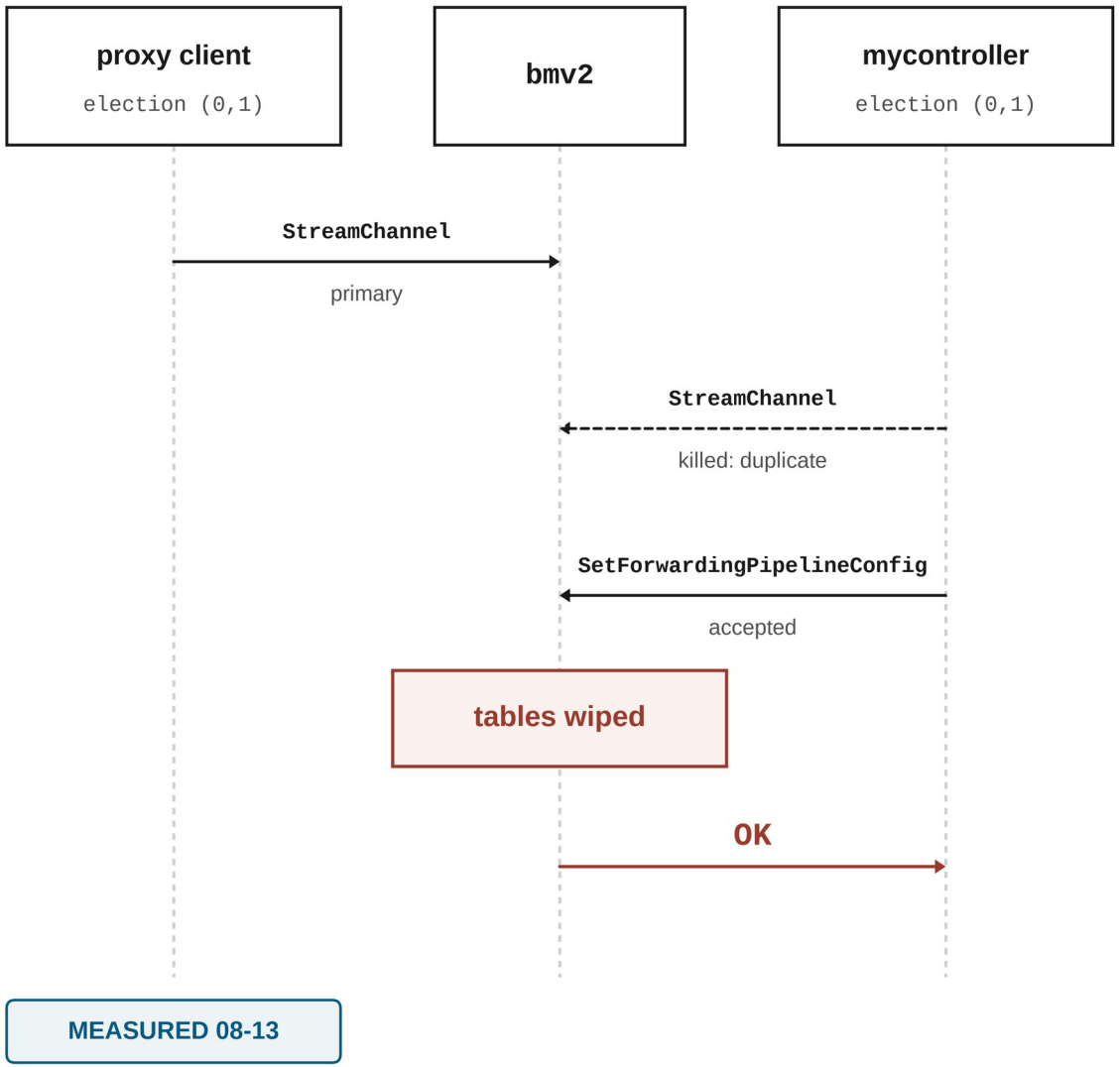
the requirement comes from	what it is	what it drives	what it found
OUTSIDE PROGRAMS	13 exercises in p4lang/tutorials — written by other people, and not for us	each exercise against 16 capability dimensions, then NDTwin against the same 16	5 real gaps · 2 findings only a foreign program exposes
OUTSIDE PEOPLE	six operators' motives — inherit, rush a paper, follow the handbook, collide, push traffic, restart	send each one at the machine and watch which untested pair of assumptions it crosses	two nights · 11 rounds · dozens of defects
the claim			
WHY IT WORKS	a test we write walks the path its author already thought of — an outside requirement walks somewhere else, and that is where the defects live		

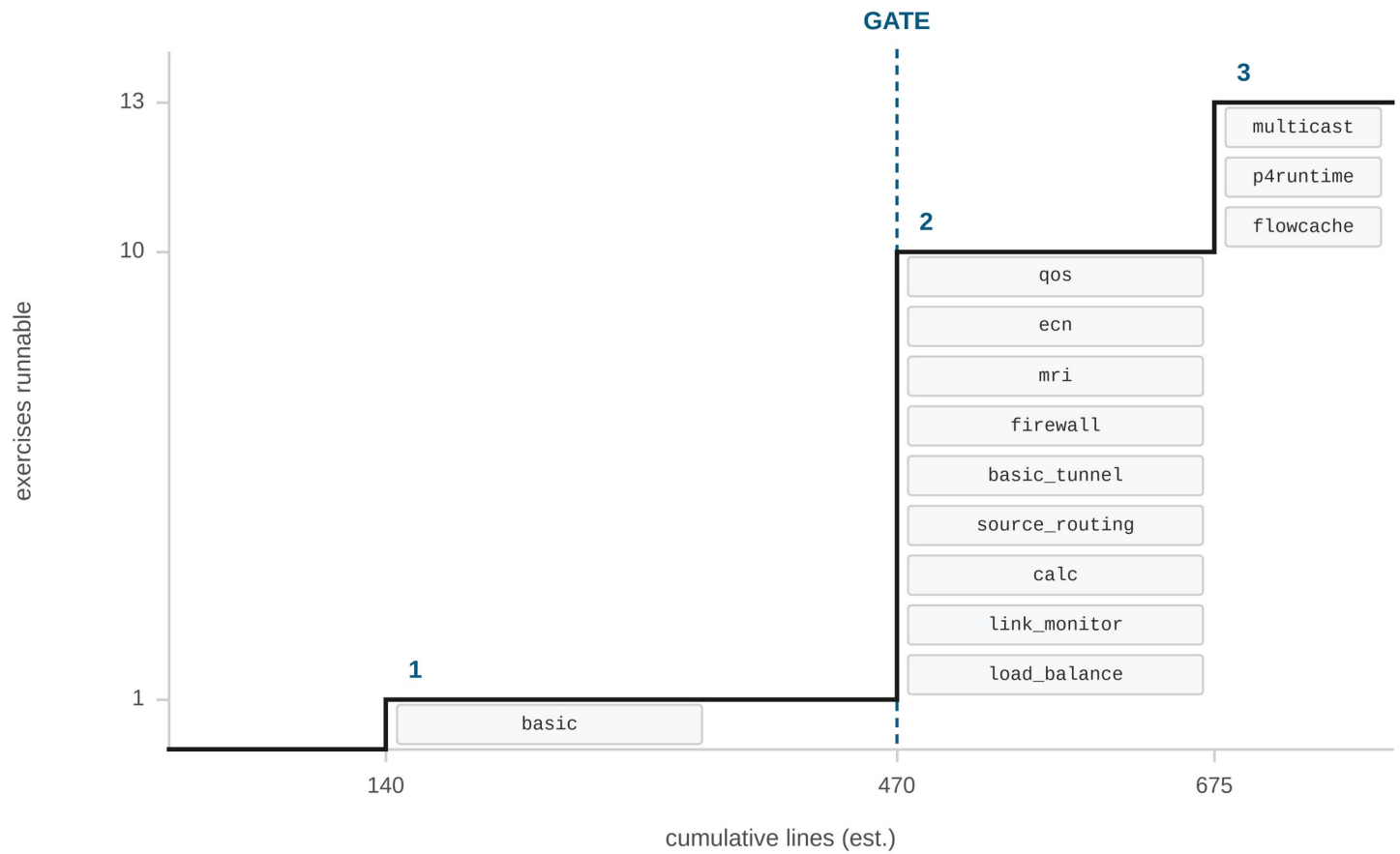
What 13 P4 exercises need vs. NDTwin today




EXERCISE
topology.json
its .p4
sX-runtime.json
mycontroller.py
KERNEL
PROXY
northbound API
topology model
G2 ~85
sFlow parser + FlowKey
G6 ~30+250
sFlow
packet-in decoder → sFlow emitter
G1 ~40
pipeline loader
G4 ~150
P4Runtime client
table writer
G5 ~180
election_id
G3 ~15
bmv2 clone
Mininet fabric · bmv2 × N
one p4info today
phase 1
phase 2
phase 3


bmv2 clone 1/256
packet_in header
fields 1..5 by position
emitter reads id 5
= sampling_rate
foreign .p4
fields renumbered
== 0 ?
no
sFlow → kernel
yes
sample dropped · no log
READ-CODE


proxy client
election (0,1)
bmv2
mycontroller
election (0,1)
StreamChannel
primary
StreamChannel
killed: duplicate
SetForwardingPipelineConfig
accepted
tables wiped
OK
MEASURED 08-13

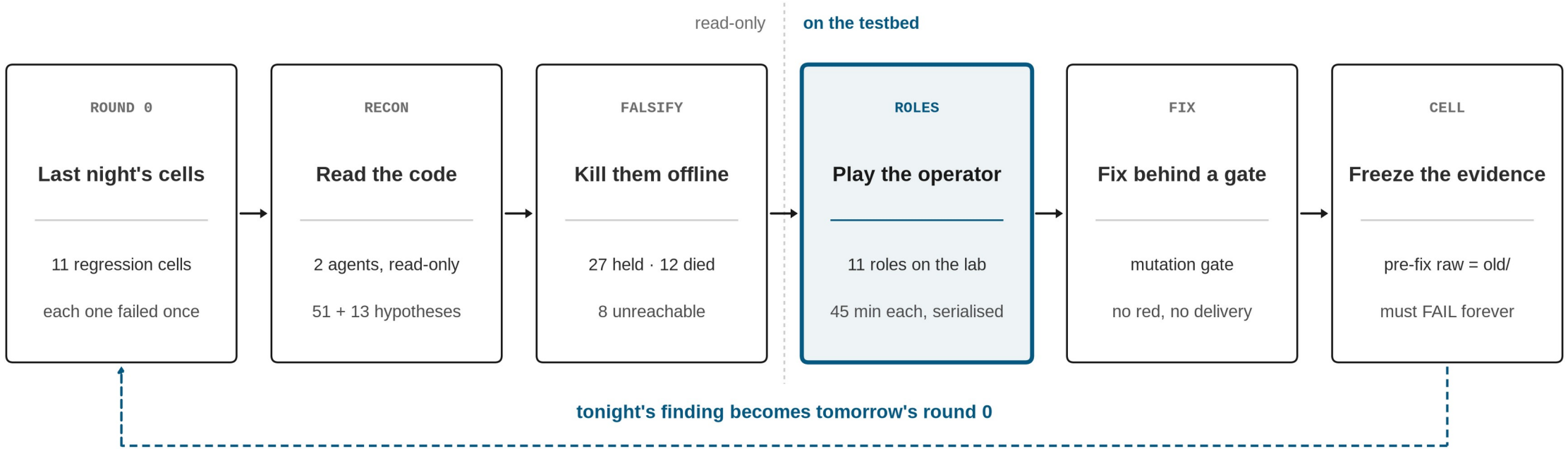


4 runs · 09-08 · tutorials harness

	solution	skeleton
source_routing	5/5	2/2
basic	5/5	4/4

expected fail

One night of the hunt



R7 a reconciler runs the whole night, read-only, one pass every 10 minutes

EVERY ROLE baseline first → 45 minutes → restore → **compare to baseline** | **if they differ, the round is void**

A role is two assumptions, crossed

	Time	Scale	State	Intent
The successor starts from someone else's wreckage	6	·	1 · 6 · 8	1 · 8
The restart cycler up and down, over and over, never cleaning	2 · 9	9	2	·
The grad student wants one figure, edits the topology file	·	3	·	3
The second operator starts while the first one is mid-teardown	4 · 12	·	·	·
The traffic pusher rules every second for an hour	·	5 · 10	·	·
The manual follower reads only the handbook, one step no more	·	·	·	11
The reconciler reads only, all night, beside whoever is on the lab	R7 every axis, in parallel			

DISPATCH

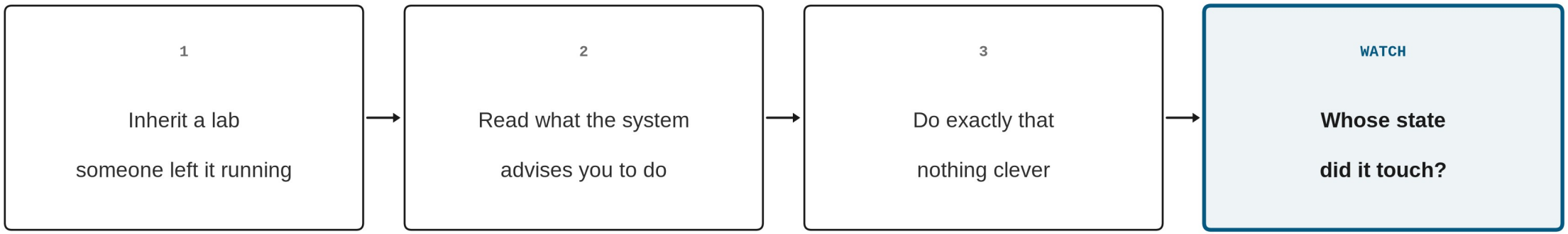
a motive, never a test case

“a motive generates action sequences you would not have thought of”

ROLE-1

The successor

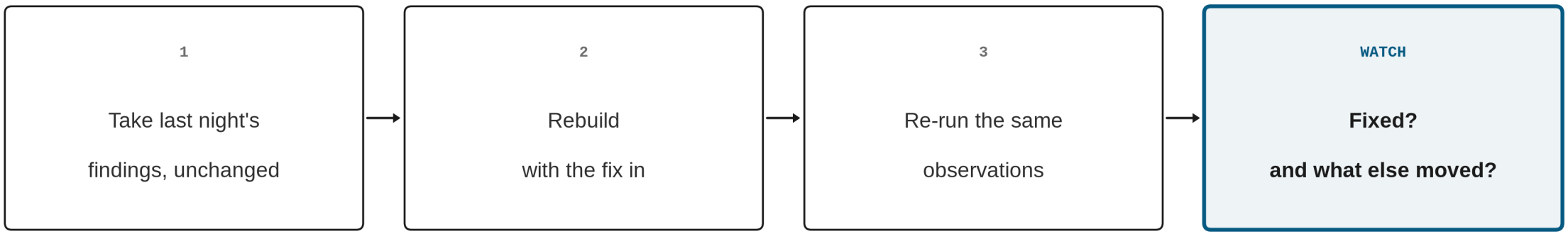
state × intent



ROLE-6 · ROLE-8

The successor, second visit

state × time



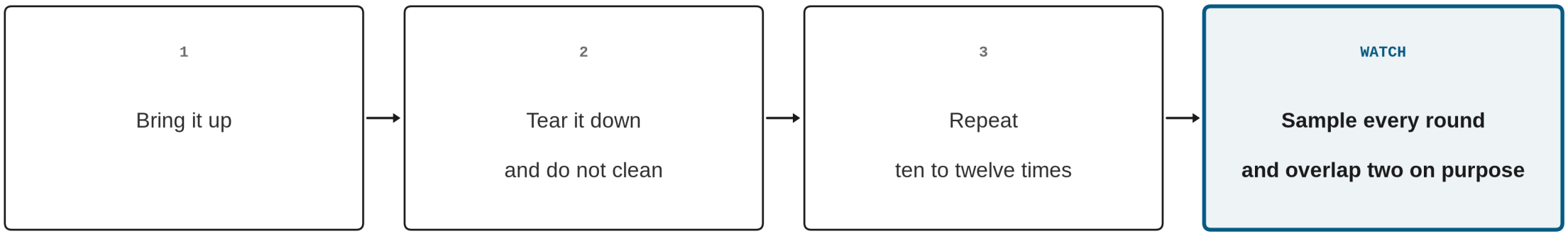
SAME METHOD, TWICE

ROLE-6 checked three findings from the restart cycler; ROLE-8 checked the successor's own

ROLE-2 · ROLE-9

The restart cycler

time × state



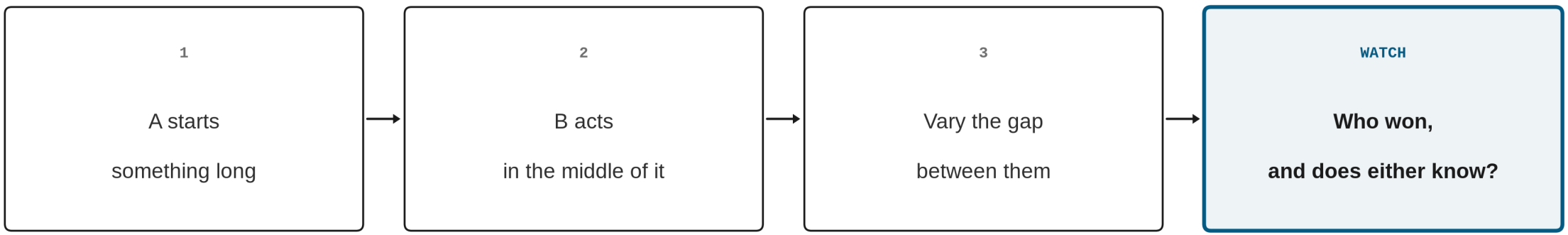
SAME METHOD, TWICE

ROLE-9 switched between 4 and 128 hosts on every cycle — 128 ran for the first time

ROLE-4 · ROLE-12

The second operator

time



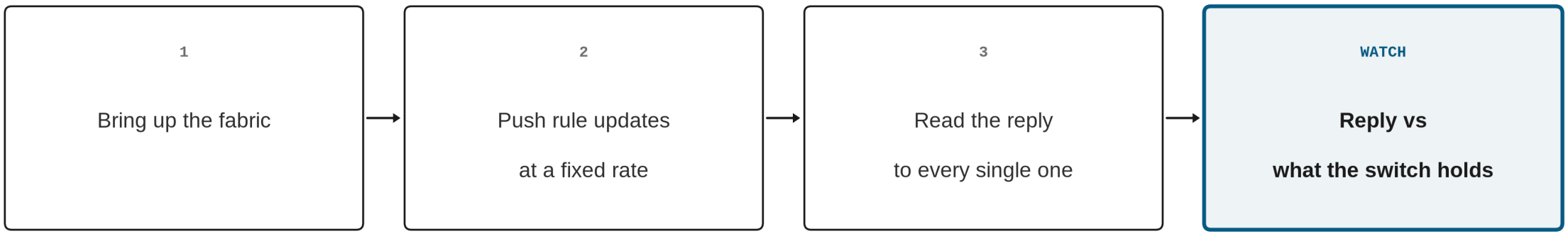
SAME METHOD, TWICE

ROLE-12 made A's long action a LIVE teardown of ten switches, not an empty lab — nobody had measured that

ROLE-5 · ROLE-10

The traffic pusher

scale



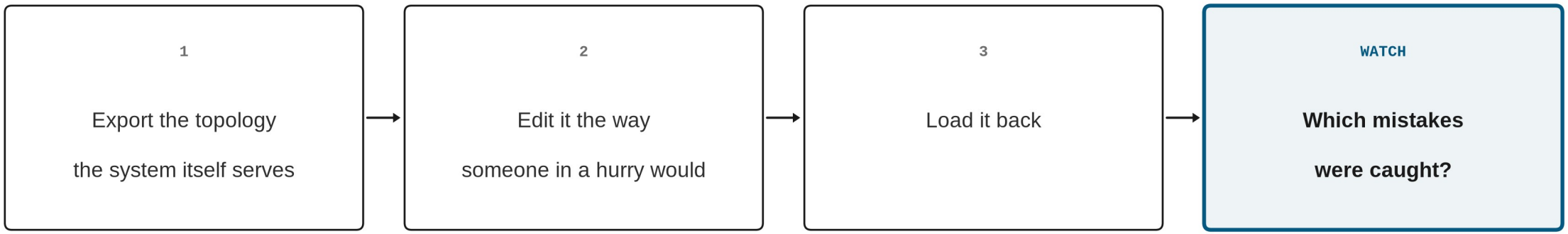
SAME METHOD, TWICE

ROLE-10 ran it on 128 hosts for a full hour — 7106 requests, 3599 seconds

ROLE-3

The graduate student

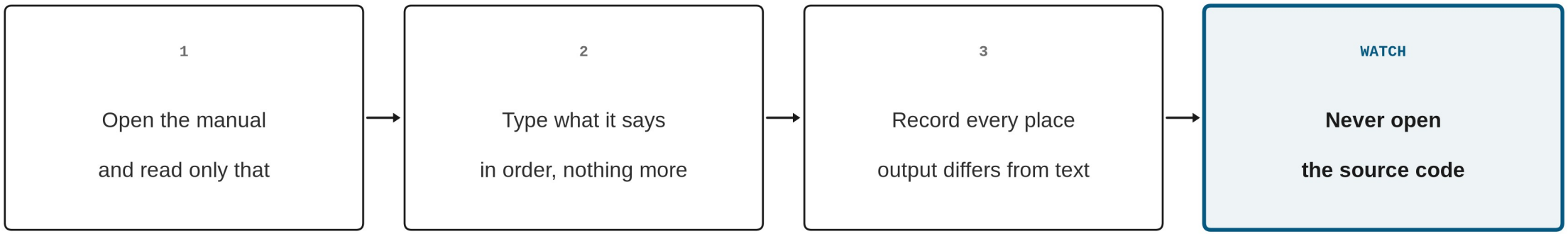
intent × scale



ROLE-11

The manual follower

intent × docs



FOUND BY THE RESTART CYCLER

The machine was empty. The twin was not.

ON THE MACHINE	
bmv2 switches	0
mininet processes	0
topology session	none

vs

ON PORT 8000

14

nodes, served on request

AND THE TWO INSTRUMENTS THAT SHOULD HAVE CAUGHT IT

`ndt status --check`	did not check
`orphans verdict`	CLEAN, no caveat

FOUND BY THE GRAD STUDENT

One field. Two doors. Opposite answers.

THE SAME FIELD

vertex_type: 2

THROUGH THE TOPOLOGY FILE

accepted

no diagnostic · enters the graph

THROUGH THE API

400

refused

the same door also took

-1	99	["10.1"]

and ["10.1"] gave two hosts one address

FOUND BY THE SECOND OPERATOR

Two people, one lab, both told yes

file keeps only the last writer

BOTH GRANTED

2 of 16

both replies said "ok lab claimed by <me>"

CORRECTLY REFUSED

0 of 9

the second one is turned away



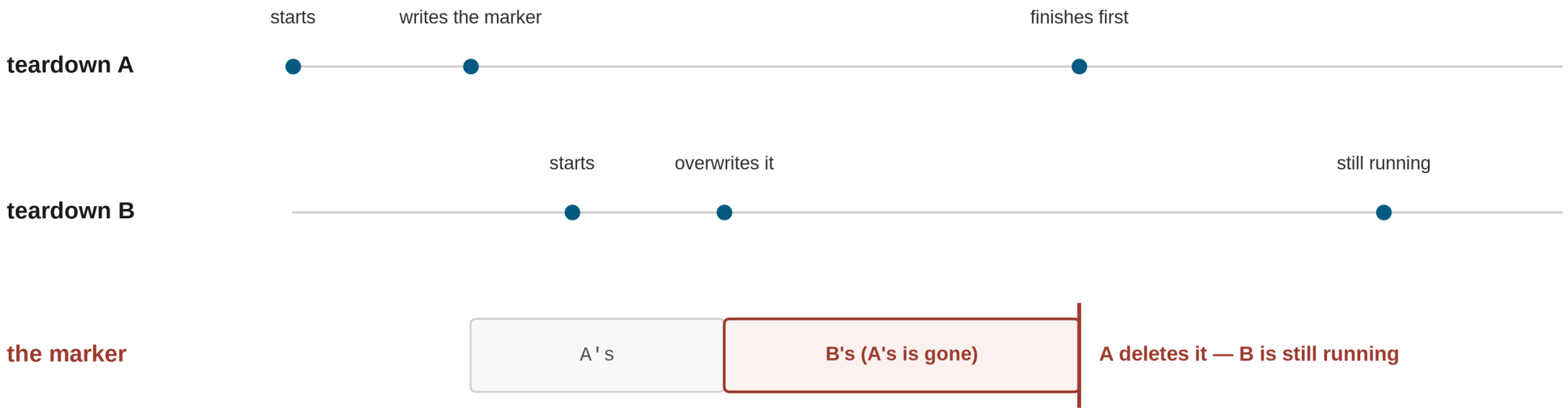
gap between the two claims

WHY

the claim is read, then written — there is no lock between the two

FOUND BY THE SECOND OPERATOR

Two teardowns, one marker, no owner



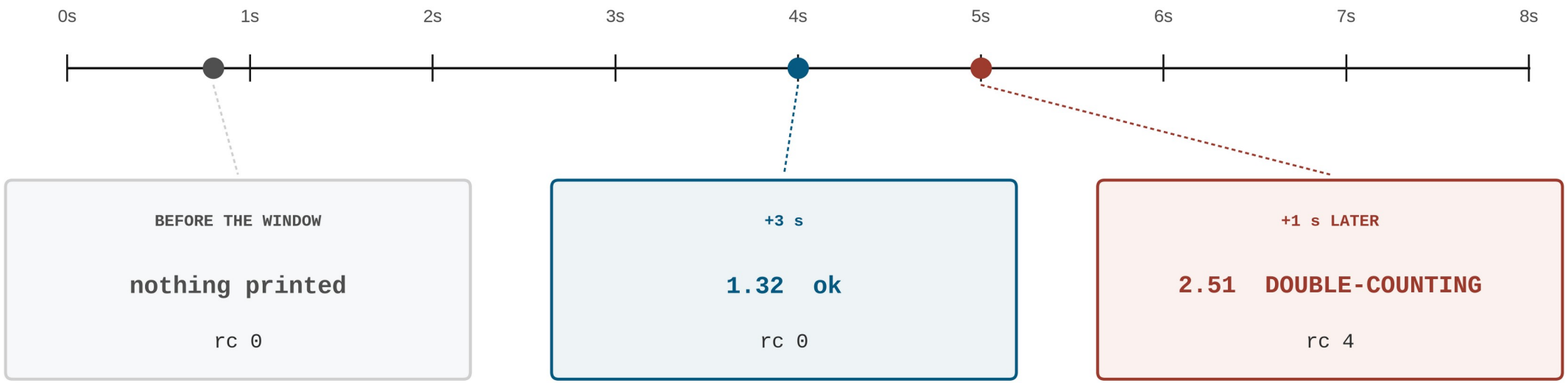
AND THEN

with the marker gone, a new bring-up is not refused — it runs to the last step

missed by seconds

FOUND BY THE TRAFFIC PUSHER

Same cut. Three verdicts. Eight seconds.



AND THE SCALE AXIS DID NOT EXIST

4 hosts

10 switches · 16 links

=

128 hosts

10 switches · 16 links

the 248 extra edges are host-facing

and every one is excluded

ONE FLAG, TWO MEANINGS

who writes is_up	what they mean by it
the power API — powerOff, after the helper confirms the process is dead	a decision: I commanded it off
the 1 Hz liveness worker, and the topology poll	an observation: it answers

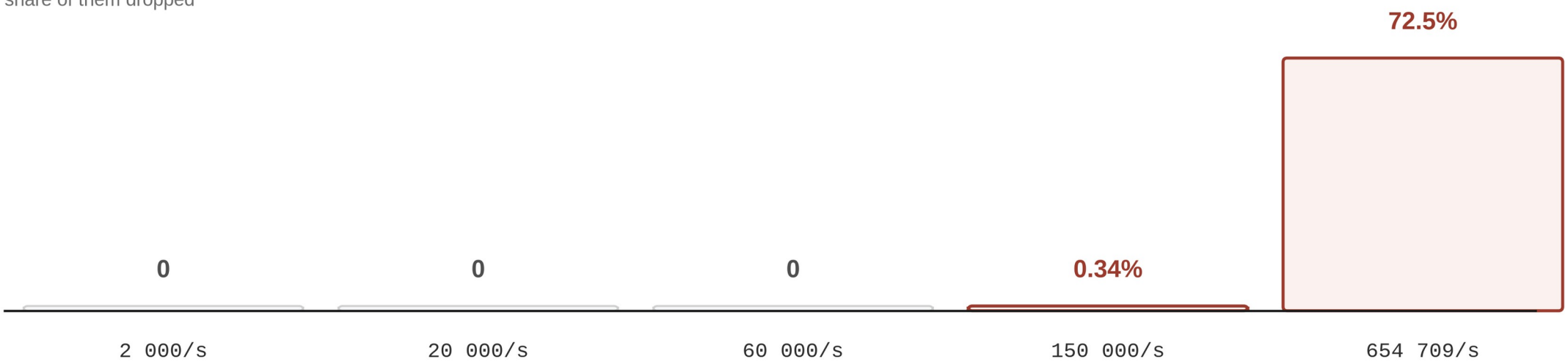
what one bit cost	measured
is_up written back to true	0.3–1.5 s after the command
and it stays true	8–13 s · 18 of 18 across both arms
two equally dead switches	commanded reads OFF · crashed reads ON
the check for “something died”	reads the same bit on both sides — it can never fire
the ruling	admin_state (command) + reachable (probe)
status	on trunk since 09-05, and published

Live two-arm run, 9 each, at 7e8d91e0 (merged 85c1a159); raw at audit-raw 19e3ab88. Ruling at b57736cd; on trunk via cdc8dad9.

RANK 2 · #53

The telemetry lied harder as the load rose

share of them dropped



samples the network offered, per second

AT EVERY RUNG — every endpoint answered success — and four drop counters existed that no API could read

AFTER THE FIX — status · loss_fraction · the numerator and the denominator, on the wire

RANK 4 · #46 / #36

You switched it off. The poll switched it back.

THE SWITCH

off, and it stays off

WHAT THE TWIN SAYS

up



HOW OFTEN — **2 runs out of 9 — the poll wrote is_up back to true**

WHY IT MATTERS — **the power command is the only way back, and it reads the flag the poll just overwrote**

RANK 7 · F-14 / F-16 / F-4

A switch died. Almost nothing died with it.

SWITCH

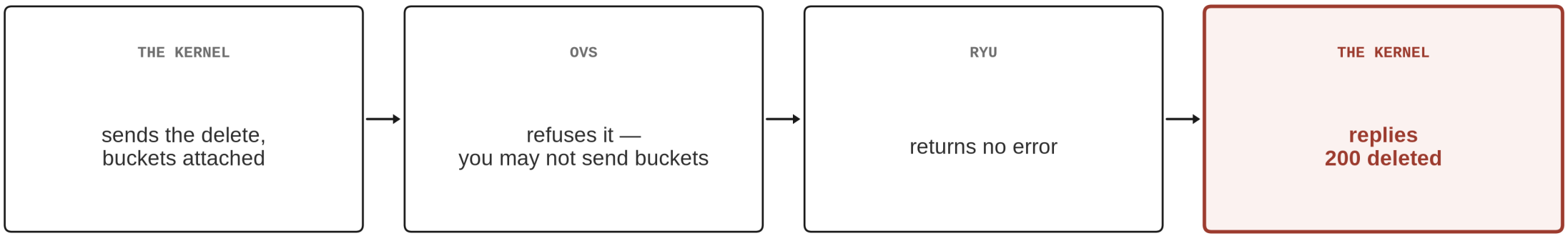
dead

the edges to other switches	**marked down**
the edges facing its hosts	**still up**
the hosts themselves	**still up**
a dead link, next poll	**brought back up**

one switch, four questions, and only the first was answered honestly

RANK 8 · #1 / A-10

Deleted, it said. The group was still there.



READ IT BACK — the group is on the switch, and its id can never be reused

THE FIX — only a decisive yes from the switch is allowed to say deleted

RANK 9 · #2 / C-4

The table had the rule 13 seconds early

WHAT THE FLOW TABLE SHOWED

the rule, from the cache

WHEN THE SWITCH ACTUALLY HAD IT

13.4 s after the install



IN BETWEEN	for 13 seconds the twin answered from a queue and called it the network
THE FIX	an entry is confirmed only by a decisive success on the southbound path

RANK 10 · F-8

272 edges were right by coincidence

EVERY EDGE REPORTS A CAPACITY OF 1 Gbit/s — IT IS THE FIELD'S INITIAL VALUE, NEVER OVERWRITTEN

272 edges

`declare 1 Gbit/s`

right — because the initial value
happens to equal the declared one

16 edges

`declare 10 Gbit/s`

wrong by 9 Gbit/s each —
one tenth of the headroom reported

AND THE POINT — **nothing in the response tells the 16 apart from the 272**

READ FROM THE FILE, JUST NOT USED — **the declared capacity was loaded from the file and written to the neighbouring field**

THE FIX — **report the declared capacity, and add a field saying where the number came from**

RANK 11 · A-7

A count that now says what it counts

BEFORE

`succeeded: N`

a queued write that failed
appears in no API at all

AFTER

```
dispatcher_running: true
counters_cover: the four dispatch routes
includes_boot_time_programming: false
includes_intent_translator: false
```

WHAT THAT BUYS	**running true with zero drops is health; running false with zero drops is delivery already over**
AND WHAT WAS REFUSED	**counters for the boot-time installs — they would have been pinned at zero forever**

a counter that can never move is worse than no counter

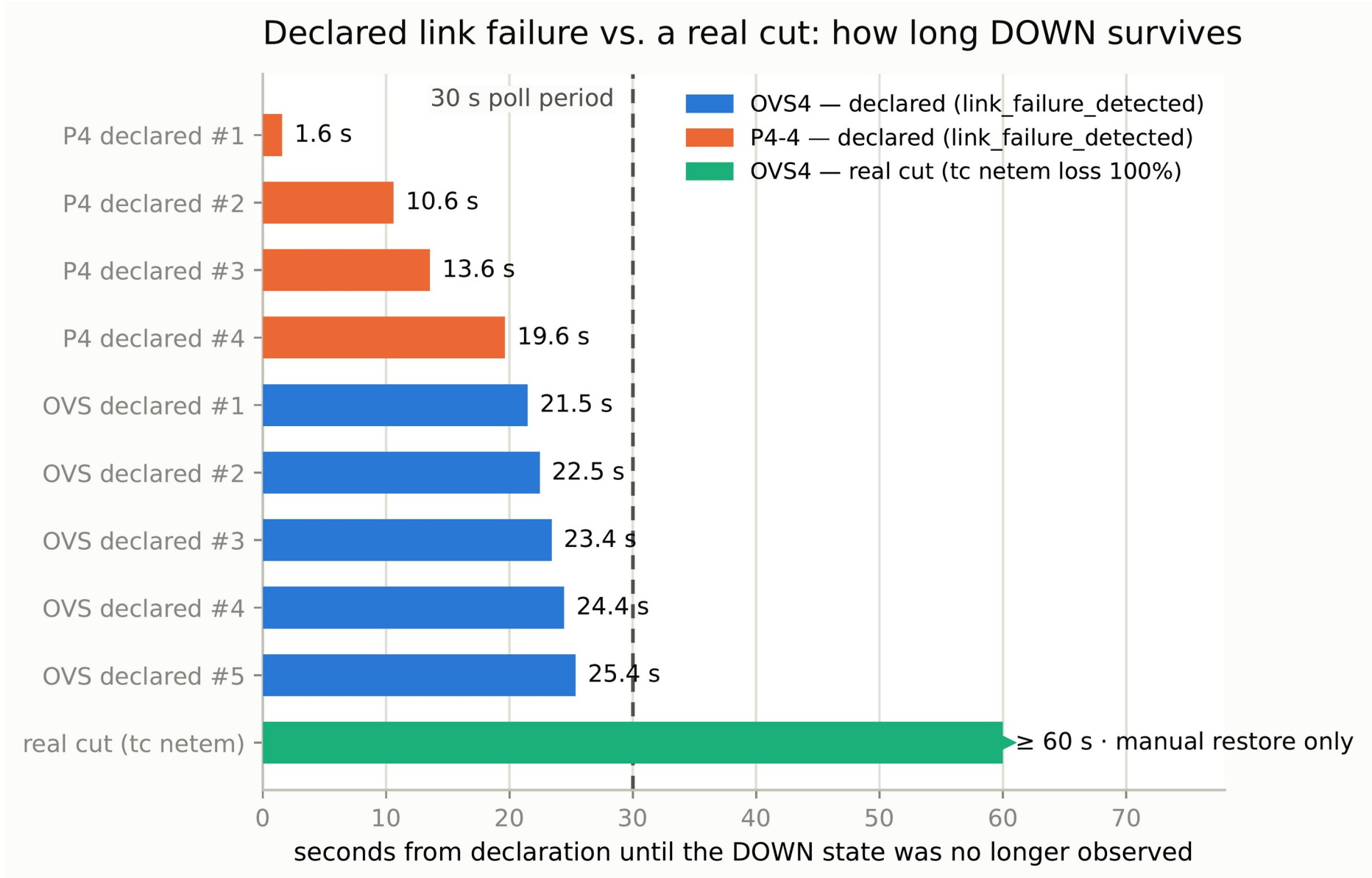
One file. Eleven doors. All of them open.

what the topology file could contain, and what the loader checked

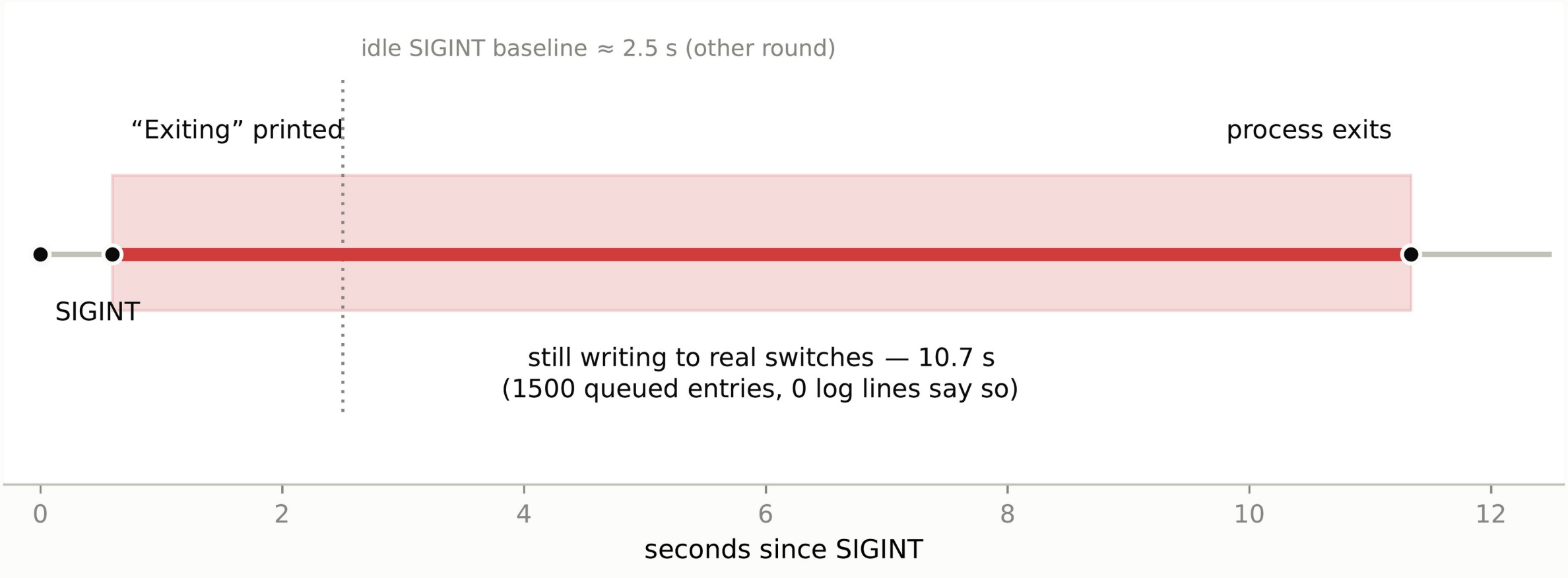
		BEFORE	NOW
an edge pointing at a node that is not there	#61 / #62	taken in silence	refused
a port or interface number out of range	#61 / #62	taken in silence	refused
a switch with an empty address	#89 / W3	taken in silence	refused
a missing bridge_name	#89 / W3	taken in silence	refused
a host declared with no address at all	B-11	taken in silence	refused
a brand_name this build cannot drive	B-12	taken in silence	refused
a vertex_type outside the range	B-14	taken in silence	refused
a link bandwidth of 0, or of -1	B-13	taken in silence	refused
a loose address that two hosts then share	B-15	taken in silence	refused
switches from two different data planes	BUG-17	taken in silence	refused
a topology with no switch in it	E-26	taken in silence	refused

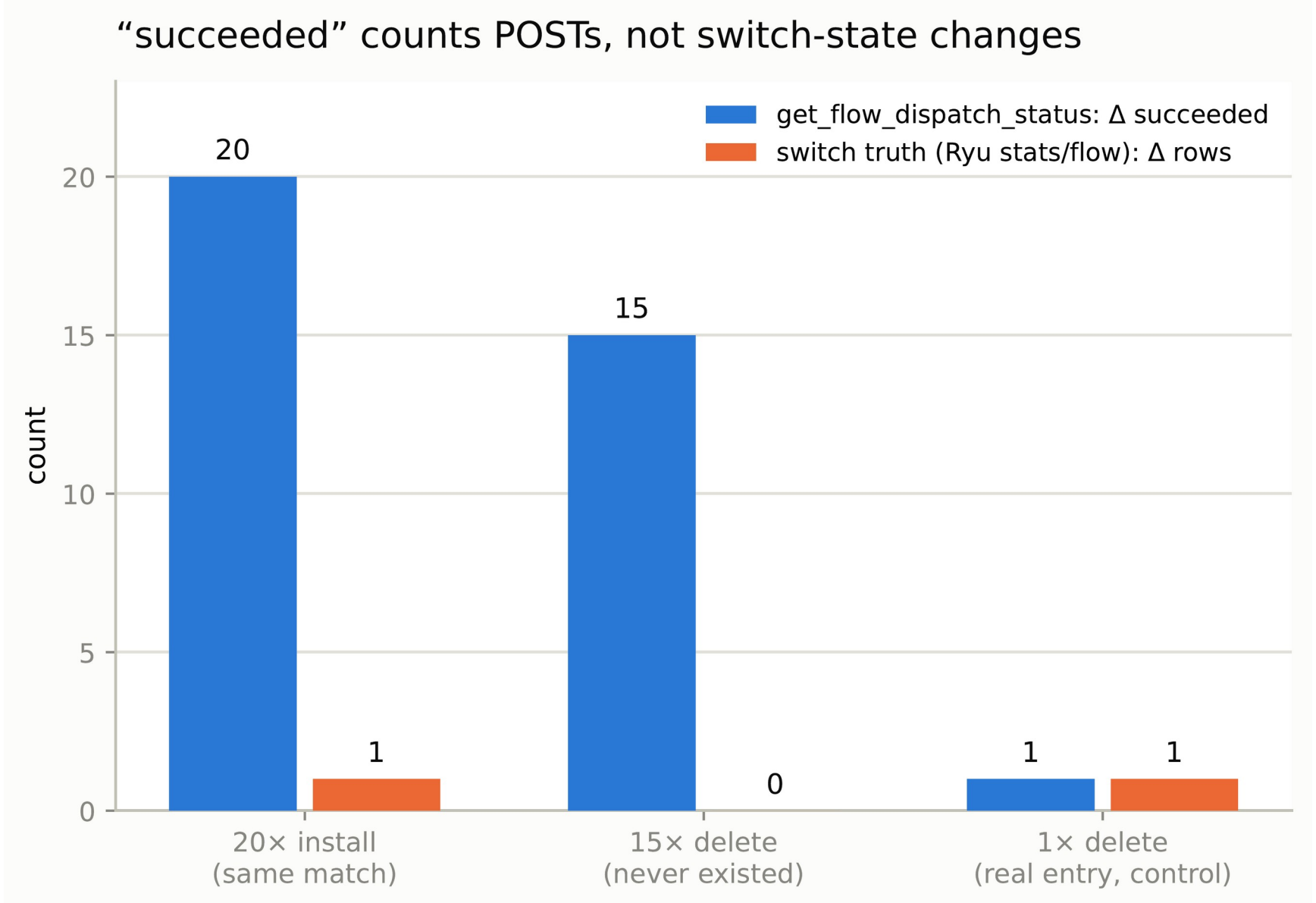
THE SHAPE — **the same value is refused when it arrives by API, and taken when it arrives by file**

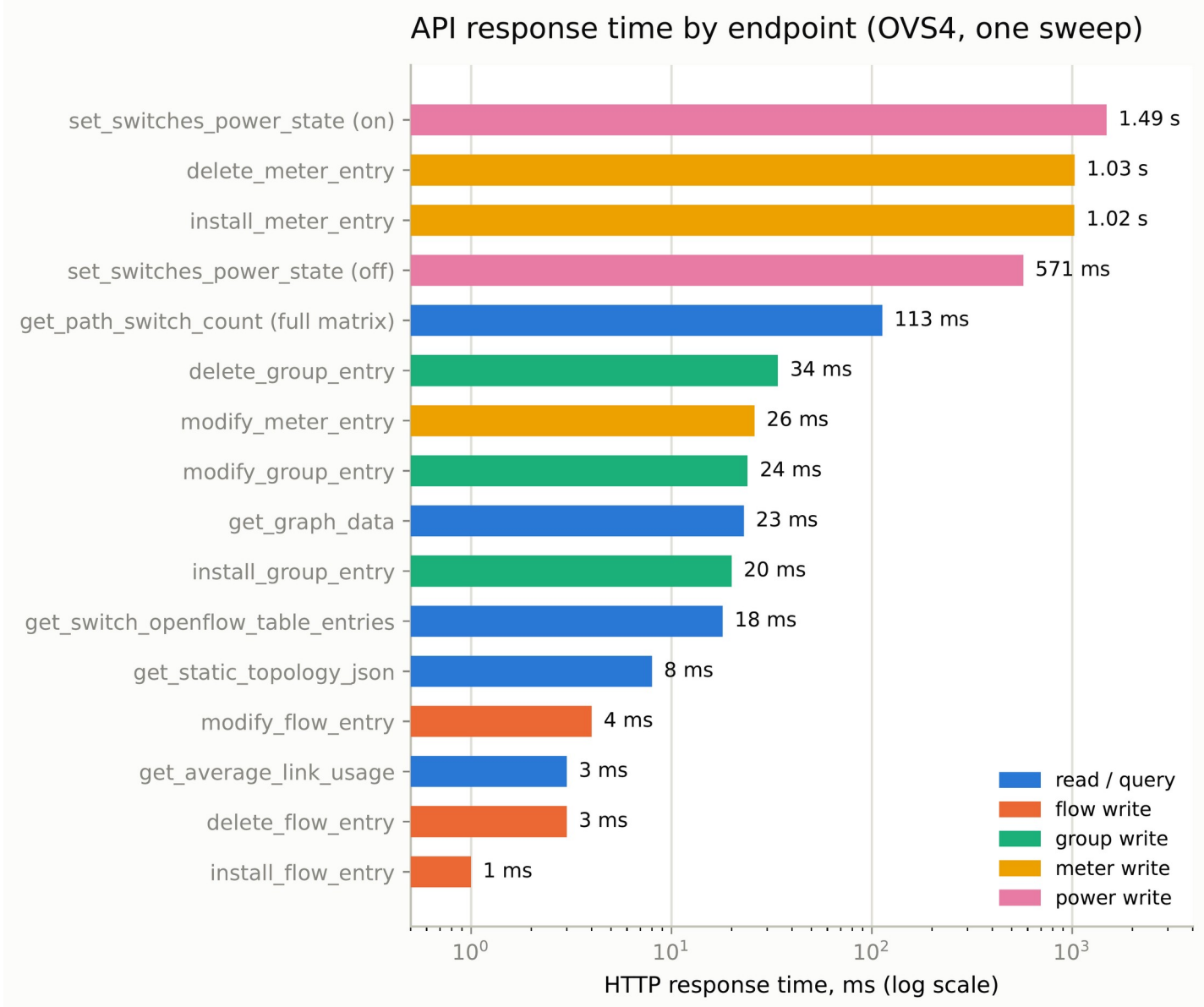
AND NOW — **the whole file is validated before the first vertex enters the graph**


Declared link failure vs. a real cut: how long DOWN survives
30 s poll period
OVS4 — declared (link_failure_detected)
P4-4 — declared (link_failure_detected)
OVS4 — real cut (tc netem loss 100%)
P4 declared #1
1.6 s
P4 declared #2
10.6 s
P4 declared #3
13.6 s
P4 declared #4
19.6 s
OVS declared #1
21.5 s
OVS declared #2
22.5 s
OVS declared #3
23.4 s
OVS declared #4
24.4 s
OVS declared #5
25.4 s
real cut (tc netem)
≥ 60 s · manual restore only
0
10
20
30
40
50
60
70
seconds from declaration until the DOWN state was no longer observed

Shutdown: southbound writes continue after “Exiting” is printed




“succeeded” counts POSTs, not switch-state changes
get_flow_dispatch_status: Δ succeeded
switch truth (Ryu stats/flow): Δ rows
count
0
5
10
15
20
20
1
15
0
1
1
20× install
(same match)
15× delete
(never existed)
1× delete
(real entry, control)


API response time by endpoint (OVS4, one sweep)
set_switches_power_state (on)
delete_meter_entry
install_meter_entry
set_switches_power_state (off)
get_path_switch_count (full matrix)
delete_group_entry
modify_meter_entry
modify_group_entry
get_graph_data
install_group_entry
get_switch_openflow_table_entries
get_static_topology_json
modify_flow_entry
get_average_link_usage
delete_flow_entry
install_flow_entry
1.49 s
1.03 s
1.02 s
571 ms
113 ms
34 ms
26 ms
24 ms
23 ms
20 ms
18 ms
8 ms
4 ms
3 ms
3 ms
1 ms
read / query
flow write
group write
meter write
power write
10^0
10^1
10^2
10^3
HTTP response time, ms (log scale)

install vs. delete readback, before and after the W1 fix

BEFORE — kernel cca5e3e4, 2026-09-04

group_id	install_group_entry	groupdesc (switch truth)	delete_group_entry (same id)
4294967295 (OFPG_ALL, reserved)	200 {"outcome": "installed"}	absent	404 {"outcome": "no_such_group"}
4294967293 (reserved)	200 {"outcome": "installed"}	absent	404 {"outcome": "no_such_group"}

AFTER — kernel 4c9e0be1, 2026-09-05

group_id	install_group_entry	groupdesc (switch truth)	delete_group_entry (same id)
4294967295 (OFPG_ALL, reserved)	502 {"outcome":"absent"} “the switch did NOT take it”	absent	404 {"outcome": "no_such_group"}
4294967293 (reserved)	502 {"outcome":"absent"} “the switch did NOT take it”	absent	404 {"outcome": "no_such_group"}
813 (control, OVS accepts)	200 {"outcome": "installed"}	present (type SELECT)	200 {"outcome": "deleted"}
899 (control, never installed)	— not attempted (modify: 404 no_such_group)	absent	— not attempted

install now says “installed” only once the switch has taken it

ndt up

bring the whole lab up, and verify it

HOW YOU CALL IT

`ndt up`	Ryu and OVS, 128 hosts — OVS is the default plane
`ndt up 4`	the same, on the P4 test bed's 4-host layout
`ndt up p4 4`	the P4 plane at 4 hosts
`ndt up p4 128`	the P4 plane at 128 hosts

WHAT ITS EXIT CODE MEANS

0	it came up and it verified
1	it measured the machine and did not like it
5	a guard refused — nothing was built
2	you called it wrong

THE SHARP EDGE

1 and 5 are different questions: 1 looked and disliked what it saw, 5 never looked

ndt down

take it all down, and prove the machine is clean

HOW YOU CALL IT

ndt down	tear down, then assert the machine is clean
ndt down --deep	also kill whatever holds any port in the table

WHAT ITS EXIT CODE MEANS

0	the teardown finished and verified clean
1	a step failed, or something is still running
3	there was nothing to tear down
5	a guard refused — nothing was torn down

THE SHARP EDGE — **3 says this teardown measured nothing — it does not say the round ended clean**

ndt status

what is running, and who has the lab

HOW YOU CALL IT

`ndt status`	what is on the machine right now
`ndt status --check`	compare it against the baseline this lab was brought up with

WHAT ITS EXIT CODE MEANS

0	every compared field matches
1	one of them does not
3	there was no baseline to compare against

THE SHARP EDGE — **with no baseline on disk the whole report is rc 3, not rc 0**

`ndt clean`

just the teardown assertion, on its own

WHAT ITS EXIT CODE MEANS

0	it had something to be about, and none of it is residue
1	residue is still there
5	a guard refused — a teardown is in flight

THE SHARP EDGE — **it refuses while another teardown is running, because that teardown is releasing the very ports it would report**

`ndt check`

the telemetry double-count tripwire

WHAT ITS EXIT CODE MEANS

0	the ratio is inside the band, or nothing was compared
4	double-counting — the tripwire fired

THE SHARP EDGE

rc 0 also means nothing was compared — the quiet answer and the healthy answer are the same number

ndt apps

start the readers and apps, and find what they left

HOW YOU CALL IT

`ndt apps`	an interactive picker, when you are on a terminal
`ndt apps <names>`	start those readers
`ndt apps orphans`	what is still running that nobody is tracking

WHAT ITS EXIT CODE MEANS

0	nothing untracked
1	untracked processes are running
2	you called it wrong

THE SHARP EDGE — **read the tally line, not the exit code — that is what the readers were taught to judge by**

`ndt ntg`

which prompt the topology hands control to

HOW YOU CALL IT

`ndt ntg cli`	hand control to the CLI
`ndt ntg prompt`	hand control to the topology's own prompt

WHAT ITS EXIT CODE MEANS

0	the choice was applied
2	you called it wrong

ndt claim

reserve the lab, so nobody tears it down under you

HOW YOU CALL IT

`ndt claim`	thirty minutes, the default
`ndt claim 75 "<note>"`	seventy-five minutes, and say what you are doing

WHAT ITS EXIT CODE MEANS

0 the lab is yours

1 somebody else holds it

2 NDT_OWNER is not set

THE SHARP EDGE — **two claims in the same millisecond were both granted, 2 times in 16 — there was no lock between the read and the write**

`ndt release`

give the lab back

WHAT ITS EXIT CODE MEANS

`0`	released
`1`	you were not holding it

THE SHARP EDGE — **the claim is a courtesy, not a lock: it stops the honest, not a command that never asks**

Found, then fixed — since 09-03

09-03			09-04	09-05	09-10		
#58	B-5/#5	#73/#74	A-11	W7	W8	W3b-3	BUG-17
P4 priority	#32	#47	#61/#62	W6	W3-3b	intent-tmpdir	E-23/25
L-9	#10/#18	#42/G-10	#38	W1/#87	W15	W15-2	E-4
G-7	sflow health	#6/#48/G-11	#82	W5/OV-1	W10	W18	E-17
G-9	#69/#70 doc	#78	#27/#76	#89/W3	W14	W17	G-13
G-6	topo round	#46/#36/#35	Q12	#88/W2	W11	W16-1/2/3	E-2
redirection	topo load	#69/#70	#1/A-10	anchor L1	W12/W13	3-51	E-29
#22/#23/#24	#4/A-12	#77	#85	anchor 68	W16	W8b	KIREF
A-14	#71	#75	#2/C-4		conventions	E-20	E-1/E-3
ndt up ovs	#17	#8			W8-8	E-21	

+7 documentation-only commits, not bricked

kernel C++ · ndt / lab tools · proxy / harness / docs

104 FIXES ON TRUNK · 0 WAITING · ALL PUBLISHED

FIXED AND SHIPPED

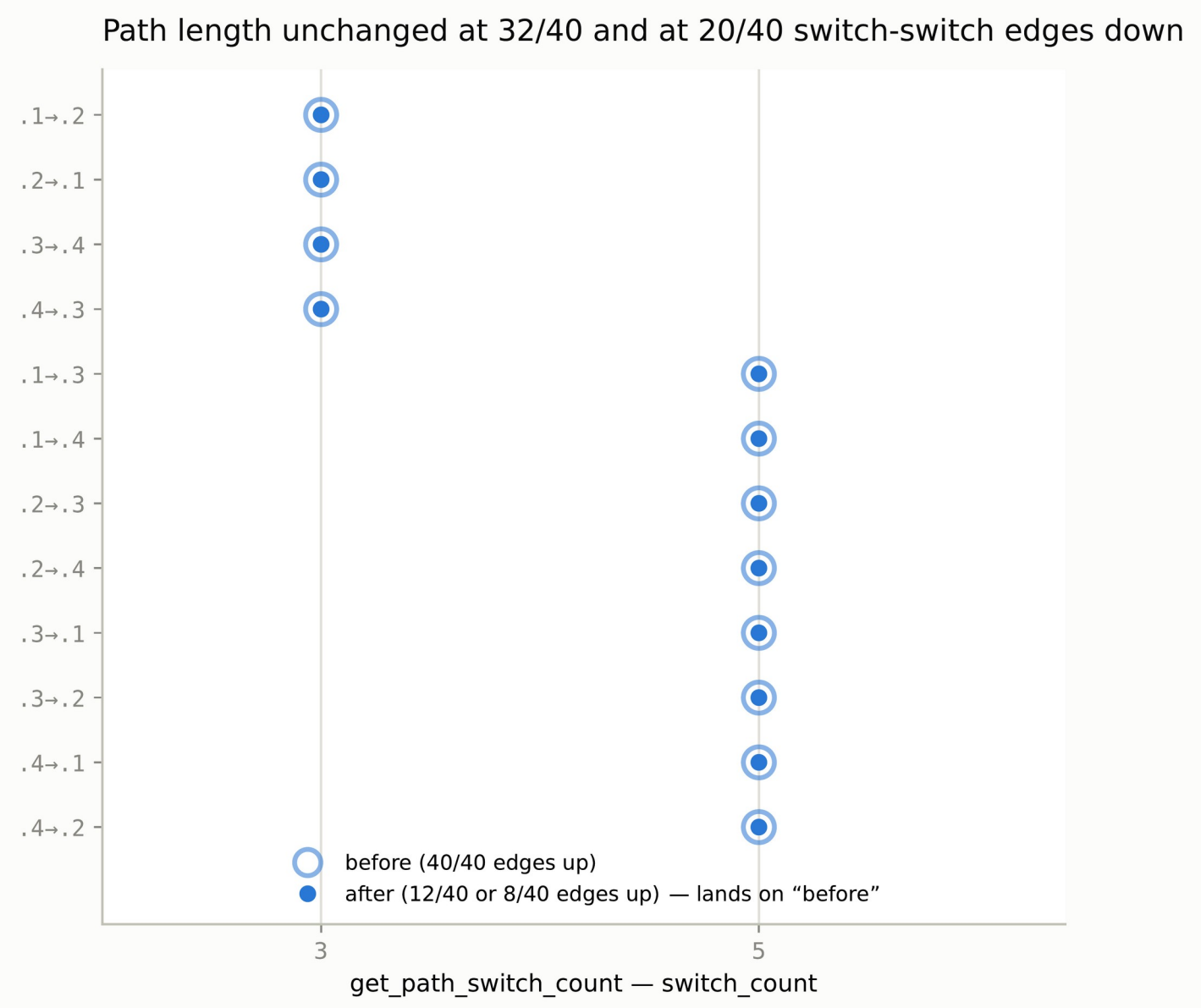
WHERE IT STANDS

104 fixes	on trunk, and on the default branch of both public repositories
0 branches	waiting — the 29 that were open on 09-08 all merged on 09-10
phase 1 · ~140 lines	unlocks basic: the first tutorial exercise that would run unchanged

Public status cross-checked two ways on 09-15: unauthenticated compare API reports behind_by 0 for both repositories. A single git ls-remote returned a stale answer and is not sufficient on its own.

WHAT THIS METHOD ADMITS ABOUT ITSELF

the limit	in its own words
reconnaissance guesses wrong a lot	hypotheses are read out of source by an agent that has run nothing · 09-11: 27 confirmed, 12 refuted, 8 unreachable
instruments certify themselves	three in one night — a miscount, a colour code splitting a token, a scan matching itself ⇒ prove the instrument can see a known thing before believing its zero
a fix can be worse than the defect	one rewrote the note to read “lab free”, so the lab declared itself empty while carrying traffic
green does not mean verified	“a grid read as the fixes are verified is worse than no grid” — the coverage table answers who is fixing, not whether it is fixed
coverage skews toward what we were told	if a whole night's findings land inside the five things he pointed at, the tests only walked his path
most role findings are not independently re-run	the live numbers are the role's own; the person filing them did not repeat the measurement
the dispatches and raw are not in version control	scratch/ is git-ignored, so this batch's tickets, reports and logs do not travel with the repository


Path length unchanged at 32/40 and at 20/40 switch-switch edges down
.1→.2
.2→.1
.3→.4
.4→.3
.1→.3
.1→.4
.2→.3
.2→.4
.3→.1
.3→.2
.4→.1
.4→.2
before (40/40 edges up)
after (12/40 or 8/40 edges up) — lands on “before”
3
5
get_path_switch_count — switch_count